

Этапы реализации Диплома по ГОСТ 19.102-77 и основные задачи на каждом этапе

1. Стадия: Техническое задание (ТЗ)

Задачи

- Описать предмет и цель: инструмент, который по YAML описанию собирает Gentoo-систему (профиль, пакеты, USE-флаги, kernel config, services, итоговый образ).
- Сформулировать требования: функциональные (парсер YAML, validator, pipeline сборки, kernel builder, package builder, artefact exporter) и нефункциональные (воспроизводимость, логирование, декларативность, безопасность, требования к среде сборки).
- Перечислить ограничения (необходимость root в некоторых операциях, зависимость от ebuild-репозитория, время сборки и требуемое дисковое пространство).
- Описать критерии приёмки (успешная сборка тестовых конфигураций, прохождение E2E, корректность kernel и USE).

Артефакты

- Документ ТЗ (цель, предмет, требования, критерии приёмки, риски).
- Набор эталонных YAML-примеров (минимальный профиль, десктоп, сервер).

2. Стадия: Эскизный проект

Задачи

- Выбрать стек: язык (например, Go/Python/Rust), YAML-парсер, инструмент для chroot/container (systemd-nspawn / docker / chroot), способ сборки ядра (genkernel / custom), метод вызова emerge (chroot + portage snapshot / overlay).
- Предложить модульную архитектуру: ConfigLoader → Validator → Planner → Executor → ArtifactBuilder → Logger → Reporter.
- Предварительные интерфейсы CLI и формат YAML (схема: metadata, profile, packages, use, kernel, filesystems, services, hooks).
- Оценка инфраструктурных требований: CI, кэш бинаров/sstate, местоположение билд-сервера.

Артефакты

- Эскизная архитектурная диаграмма и список модулей/интерфейсов.
- Прототип YAML-схемы (пример).

3. Стадия: Технический проект

Задачи

- Формализовать YAML-схему (JSON Schema / OpenAPI-like для валидации).
- Описать точные алгоритмы:
 - разрешение зависимостей пакетов с учётом USE и profile,
 - управление sstate/ccache для ускорения,
 - последовательность шагов для kernel build (config merge → make oldconfig → make) и интеграции результата,
 - план выполнения (параллелизм, порядок chroot → emerge → postinstall).
- Описать обработку ошибок и стратегию отката / безопасного прерывания сборки.
- Специфицировать форматы логов и отчётов (machine-readable + человеко-читаемые).
- Описать API/CLI команды (validate, plan, build, status, export).

Артефакты

- Техпроекта (текст/диаграммы), спецификация YAML, описание модулей и API, описание тестовой среды.

4. Стадия: Рабочий проект (реализация)

Задачи

- Реализовать модуль ConfigLoader и валидатор (на базе схемы).
- Реализовать Planner (строит DAG задач сборки).
- Реализовать Executor:
 - подготовка chroot/target (портирование stage3 или создание минимального rootfs),
 - применение profile и USE, синхронизация портежа или работа с snapshot,
 - сборка пакетов (emerge) с нужными ключами,
 - сборка ядра по указанной конфигурации,
 - postinstall hooks (systemd unit files, fstab, network),

- сборка артефакта (tarball/rootfs/iso) .
- Логирование и прогресс-бар/статусы .
- Написать тесты:
 - unit (парсер, validator) ,
 - интеграционные (сборка мини-конфига в контейнере) ,
 - E2E (сравнение результата с ожидаемым набором пакетов/ядра) .
- Подготовить CI-pipeline (GitHub Actions / GitLab CI) для автоматической проверки и сборки контейнера .
- Подготовить документацию: руководство пользователя, примеры YAML, руководство разработчика .

Артефакты

- Исходники, CI скрипты, тесты, бинарные релизы/образ Docker, документация .

5. Стадия: Ввод в действие (передача, демонстрация, обучение)

Задачи

- Подготовить демонстрационные кейсы: минимальная система, desktop, кастом-ядро .
- Провести сборку в контролируемой среде и представить результаты (лог, артефакт, diff конфигов) .
- Подготовить пакет для распространения: installer/binaries, Docker image, инструкции по использованию .
- Подготовить презентацию/демо для защиты, примеры конфигураций, чек-лист приёмки .
- Сбор отзывов и план дальнейших улучшений (sstate-sharing, GUI, поддержка overlay) .

Артефакты

- Релиз, демонстрационные образы, обучающие материалы и отчёт о вводе в действие .